



CLI Cheatsheet - New Features

Overview

Since Nats-server v2.12 and 2.14 a number of features have been added.

Some have not (yet) been added to the standard API but are published as **orbit extensions**.

- <https://github.com/synadia-io/orbit.go>
- <https://github.com/synadia-io/orbit.java>
- <https://github.com/synadia-io/orbit.net>
- <https://github.com/synadia-io/orbit.js>
- <https://github.com/synadia-io/orbit.rs>
- <https://github.com/synadia-io/orbit.c>

New stream capabilities

See usage detailed examples in the following sections.

Per-Message TTL - Allows for auto cleaning caches and auto removal of delete markers in KVs.

```
nats s add S1 --defaults --subjects S1 --allow-msg-ttl
```

Atomic Counters in KV stores. Works across sources

```
nats s add S2 --defaults --subjects S2 --allow-counter
```

Scheduled messages - Send messages with delay or on a repeated schedule

```
nats s add S3 --defaults --subjects S3.* --allow-schedules  
--allow-msg-ttl
```

Atomic batch - Commit a message batch to a stream atomically. Batch size is limited

```
nats s add S4 --defaults --subjects S4 --allow-batch
```

Fast batch - Send messages in guaranteed sequence without waiting for individual acks.

```
nats s add S5 --replicas=3 --defaults --subjects S5 --allow-fast
```

Warning: Only activate features on streams when you intend to use them.

- Some capabilities cannot be disabled once enabled (e.g. schedules)
- Some capabilities are incompatible with Key Value Store or Source and Mirror (e.g. schedules)

Message TTL

Publish a message with different TTLs - see them disappear

```
nats pub S1 Hi --header=Nats-TTL:5s  
nats pub S1 Hi --header=Nats-TTL:10s  
nats pub S1 Hi --header=Nats-TTL:15s  
watch -n 1 'nats s info S1'
```

Atomic Counters

Count (add) to a value atomically. Works on regular streams as well as KVs.

```
nats request S2 '' --header=Nats-Incr:+1  
nats request S2 '' --header=Nats-Incr:+2  
nats stream get S2 --last-for=S2'
```

Scheduled Messages

Schedule messages to be sent later.

- All scheduling happens in the same stream.
- The schedule lives in **unique** subject it is sent on. The schedule can be deleted, overwritten etc.
- The message is sent to **--schedule-dest=SUBJECT**
- The schedule may copy the last message from **--schedule-source=SUBJECT**

Schedule **once** with delay.

```
nats pub S3.1 Hi --schedule-dest=S3.s1 --schedule-at="$(date -d '+10  
seconds' -Iseconds)"  
nats stream get S3 --last-for=S3.s1'
```

Scheduled message has a TTL.

```
nats pub S3.2 Hi --schedule-ttl=10s --schedule-dest=S3.s2  
--schedule-at="$(date -d '+10 seconds' -Iseconds)"  
nats stream get S3 --last-for=S3.s2'
```

Schedule **cron** style

```
nats pub S3.3 Hi --schedule-dest=S3.s3 --schedule-cron='*/5 * * * * *'  
nats stream get S3 --last-for=S3.s3'
```

Schedule last message from subject

```
nats pub S3.template Hello'  
nats pub S3.4 Hi --schedule-dest=S3.s4 --schedule-cron='*/5 * * * * *'  
' --schedule-source=S3.template  
nats stream get S3 --last-for=S3.s4'
```

Atomic Batch Publish

Publish multiple messages in one atomic, all or nothing, operation. Limited to the same stream.

```
echo -e 'A\nB\nC\n' | nats pub --atomic --send-on=newline S4
```

Fast Batch Publish

Allows safe (no dups, no messages lost) fast publishing in order.

```
nats bench js pub fast S5 --stream S5
```

Stream I/O buffer optimizations

Replicated streams do NOT flush their buffers to disk immediately, but rather with a delay or when **sync_interval** triggers. With today's server and OS reliability in memory replication is sufficient guarantee for data persistence over a few seconds interval.

Non replicated stream will flush to disk by default. If performance wins over data consistency replicas=1 streams can be set to async mode

```
nats s add S6 --replicas=1 --defaults --subjects S6  
--persist-mode=async
```



CLI Cheatsheet - New Features

Check stream downgrade compatibility

Check the compatibility of streams against past Jetstream API levels. Depends on the features actually activated. The command below will show the required API level of all streams.

```
nats server report downgrade 0
```

Consumer pause, resume and reset

Consumers can now be paused (for an amount of time), resumed or reset to their initial state or a sequence number.

```
nats consumer create S1 C1 --defaults --pull
```

```
nats consumer pause S1 C1 10s
```

```
nats consumer resume S1 C1
```

You cannot reset a consumer to a sequence number earlier than the one it started on. But you can set a future one, skipping messages for a while.

```
nats consumer reset S1 C1
```

```
nats consumer reset S1 C1 --sequence=42
```

E.g. you can now pause a stuck consumer, find the sequence number of the message it fails to process and then skip over it.

Source/Mirror with durable consumers

Previously source/mirror relied on ephemeral consumers to initiate message transport, with replication state stored on the mirror stream only.

Ephemeral consumers do not work as intended on interest and workqueue streams and do not allow control to the operator controlling the stream from which messages originate.

2.14 now supports a new flow_control consumer which can be explicitly configured to be used with source and mirror streams.

Create Origin stream with a durable consumer. The consumer is a push consumer, so we need to set a delivery subject.

```
nats s add S_ORIGIN --defaults --subjects S_ORIGIN
```

```
nats consumer create --defaults --ack=flow_control --heartbeat=1s  
--deliver=all S_ORIGIN C_MIRROR --target mirror.S_ORIGIN.C_MIRROR
```

Create stream. Mirror config accepts a JSON fragment. The permutation of configuration options have become too complex to support as command line parameters.

```
nats s create S_MIRROR --defaults --mirror='{ "name": "S_ORIGIN",  
"consumer": { "name": "C_MIRROR",  
"deliver_subject": "mirror.S_ORIGIN.C_MIRROR" } }'
```